RateMedia — Documentação do Projeto

Arthur Gaspare Camzano

12/11/2026

Sumário

1. Visão Geral do Projeto
   1.1 Propósito
   1.2 Problema
   1.3 Objetivos
   1.4 Escopo
   1.5 Público-alvo e atores
   1.6 Stack tecnológica
   1.7 Glossário
2. Requisitos
   2.1 Requisitos funcionais
   2.2 Requisitos não funcionais
   2.3 Regras de negócio
3. Casos de Uso
   3.1 Diagrama de casos de uso
   3.2 Lista de casos de uso
   3.3 Especificações detalhadas
4. Histórias de Usuário
   Épico E1 — Conta e acesso
   Épico E2 — Descoberta
   Épico E3 — Biblioteca e avaliações
   Épico E4 — Listas e perfil
   Épico E5 — Campanhas de RPG
   Épico E6 — Administração
   Épico E7 — DevOps e documentação
   Resumo do backlog
5. Arquitetura
   5.1 Visão geral
   5.2 Camadas
   5.3 Estrutura de pastas
   5.4 Mapa de navegação
   5.5 Decisões arquiteturais (ADR resumidas)
   5.6 Fluxo de autenticação e sessão
6. Modelo de Dados
   6.1 Diagrama Entidade-Relacionamento (DER)
   6.2 Tipos enumerados
   6.3 Dicionário de dados
   6.4 Funções e triggers
   6.5 Políticas de Row Level Security
   6.6 Migrações
7. Diagrama de Classes
   7.1 Classes de domínio
   7.2 Integração externa
   7.3 Serviços e utilitários

7.4 Diagramas de estado
8. Diagramas de Sequência
8.1 Cadastro e login (UC01, UC02)
8.2 Avaliar e resenhar obra (UC05, UC06)
8.3 Adicionar obra a uma lista (UC08)
8.4 Criar, avaliar e gerenciar campanha (UC11, UC12)
8.5 Importar obra de API externa (UC14)
8.6 Moderação e papéis (UC15, UC16)
9. Server Actions e Integrações
9.1 Server Actions
9.2 Respostas tipadas (listas)
9.3 APIs externas
9.4 Variáveis de ambiente
10. DevOps: CI/CD, Docker e Deploy
10.1 Diagrama de implantação
10.2 Integração contínua (GitHub Actions)
10.3 Entrega contínua (Vercel)
10.4 Docker
10.5 Execução local
10.6 Fluxo de trabalho Git
10.7 Segurança operacional
11. Estratégia de Testes
11.1 Pirâmide
11.2 Suítes
11.3 Casos de teste manuais (aceitação)
11.4 Critérios de qualidade

1. Visão Geral do Projeto

Item	Descrição
Projeto	RateMedia
Versão	0.2.0
Disciplina	DevOps — Fatec
Tipo	Aplicação web (SaaS social)
Repositório	`LuisGlauser/RateMedia`
Equipe	Luís Felipe Glauser Lucas, Felipe Ferreira de Souza, gabrieldasilva88188, Arthur Gaspare Camzano
Documentação	Arthur Gaspare Camzano

1.1 Propósito

O **RateMedia** é uma aplicação web no estilo *Letterboxd* para **rastrear, avaliar e resenhar animes, mangás, séries e filmes** em um único lugar. O diferencial é o módulo de **campanhas de RPG de mesa**, que são criadas, divulgadas e avaliadas pela própria comunidade.

1.2 Problema

Fãs de cultura pop usam várias plataformas diferentes (MyAnimeList para animes, Letterboxd para filmes, planilhas para séries) e não há um espaço que reúna também as campanhas de RPG. Isso fragmenta o histórico do usuário e dificulta a descoberta de conteúdo.

1.3 Objetivos

1. Centralizar o acompanhamento (status e progresso) de quatro tipos de mídia.
2. Permitir notas (1–10 na interface, 0–5 no banco) e resenhas com marcação de *spoiler*.
3. Oferecer listas personalizadas, públicas ou privadas.
4. Criar uma vitrine comunitária de campanhas de RPG com avaliações.
5. Dar aos administradores ferramentas de curadoria (CRUD, importação de APIs externas), moderação e gestão de papéis.
6. Aplicar práticas de DevOps: CI no GitHub Actions, deploy contínuo na Vercel, imagem Docker e segurança por RLS.

1.4 Escopo

Dentro do escopo: cadastro/login por e-mail e senha; busca avançada com filtros, ordenação e paginação; página de obra; biblioteca pessoal; listas; perfil público; campanhas de RPG; painel administrativo; importação de metadados do AniList, Jikan e TMDB; documentação (esta).

Fora do escopo (nesta versão): login social, notificações em tempo real, chat, aplicativo móvel nativo, recomendações por IA, upload de imagens (as capas são URLs).

1.5 Público-alvo e atores

Ator	Descrição
Visitante	Navega sem conta: vê a home, busca, obras e campanhas.
Usuário autenticado	Gerencia biblioteca, notas, resenhas, listas, perfil e campanhas próprias.
Administrador	Usuário com `role = 'admin'`: cura o catálogo, modera conteúdo e gerencia papéis.
APIs externas	AniList, Jikan (MyAnimeList) e TMDB fornecem metadados das obras.

1.6 Stack tecnológica

Camada	Tecnologia
Front-end	React 19, Next.js 15 (App Router, Server Components, Server Actions), Tailwind CSS 3, lucide-react
Back-end	Server Actions do Next.js + Supabase (PostgREST)
Banco de dados	PostgreSQL (Supabase) com Row Level Security
Autenticação	Supabase Auth (e-mail/senha) via `@supabase/ssr` com cookies
Integrações	AniList GraphQL, Jikan REST v4, TMDB REST v3
Testes	Vitest
Qualidade	ESLint, TypeScript (`tsc --noEmit`)
CI/CD	GitHub Actions + Vercel
Contêiner	Docker (multi-stage, Next standalone) + Docker Compose

1.7 Glossário

Termo	Significado
Obra / Media	Item do catálogo: anime, mangá, série ou filme.
Entrada de biblioteca	Registro do usuário para uma obra (status, progresso, nota, resenha).
Lista	Coleção ordenada de obras criada pelo usuário.
Campanha	Mesa de RPG divulgada por um mestre, com sistema, sinopse e regras.
RLS	*Row Level Security*: políticas do PostgreSQL que limitam quais linhas cada usuário lê/escreve.

Termo	Significado
Server Action	Função assíncrona executada no servidor, chamada diretamente por formulários.
Status editorial	Estado de publicação da obra: publicado, rascunho ou em revisão.
Unidades	Total de episódios (anime/série), capítulos (mangá) ou 1 (filme).

2. Requisitos

2.1 Requisitos funcionais

ID	Requisito	Prioridade	Ator
RF01	Cadastrar conta com username, e-mail e senha (mín. 6 caracteres).	Alta	Visitante
RF02	Autenticar com e-mail e senha e redirecionar para a rota de origem (`next`).	Alta	Visitante
RF03	Encerrar sessão.	Alta	Usuário
RF04	Buscar obras e campanhas por texto e tipo.	Alta	Todos
RF05	Filtrar a busca por faixa de nota e gêneros.	Média	Todos
RF06	Ordenar a busca (mais bem avaliados, populares, recentes, A–Z) e paginar (12 por página).	Média	Todos
RF07	Exibir a página da obra com sinopse, metadados, média de notas e críticas da comunidade.	Alta	Todos
RF08	Definir status da obra na biblioteca (Quero assistir/ler, Assistindo/Lendo, Concluído, Abandonado).	Alta	Usuário
RF09	Registrar progresso (episódios/capítulos).	Média	Usuário
RF10	Avaliar a obra com nota de 1 a 10.	Alta	Usuário
RF11	Escrever resenha e marcar se contém spoilers.	Alta	Usuário
RF12	Visualizar “Minha biblioteca” agrupada por status.	Alta	Usuário
RF13	Criar, visualizar e excluir listas (públicas ou privadas).	Média	Usuário
RF14	Adicionar/remover obras de listas a partir da página da obra; criar lista já com a obra.	Média	Usuário

ID	Requisito	Prioridade	Ator
RF15	Editar perfil (bio, localização, avatar e banner).	Baixa	Usuário
RF16	Listar campanhas de RPG com nota média e quantidade de avaliações.	Alta	Todos
RF17	Criar campanha (título, sistema, sinopse, regras, status, capa).	Alta	Usuário
RF18	Editar e excluir campanha (apenas autor ou admin).	Alta	Usuário/Admin
RF19	Avaliar campanha (nota 0–5 e comentário), uma avaliação por usuário.	Alta	Usuário
RF20	Painel admin com visão geral (métricas).	Média	Admin
RF21	CRUD do catálogo de obras com status editorial.	Alta	Admin
RF22	Importar obras do AniList/Jikan (anime/mangá) e TMDB (filme/série) sem duplicar.	Alta	Admin
RF23	Moderar resenhas (limpar texto preservando a nota) e excluir avaliações de campanhas.	Alta	Admin
RF24	Promover/rebaixar usuários entre `user` e `admin` (exceto a si mesmo).	Média	Admin
RF25	Disponibilizar documentação do projeto em `/docs` (Markdown, diagramas, Word e PDF).	Média	Todos

2.2 Requisitos não funcionais

ID	Categoria	Requisito
RNF01	Segurança	Todas as tabelas com RLS habilitada; escrita apenas pelo dono do registro ou admin.
RNF02	Segurança	Credenciais somente em variáveis de ambiente; `.env.local` fora do Git; nenhuma chave `service_role` no front-end.

ID	Categoria	Requisito
RNF03	Segurança	Ações administrativas revalidadas no servidor (`requireAdmin()`), além da RLS.
RNF04	Segurança	Entradas sanitizadas: `trim`, limites de tamanho, URLs só `http(s)`, UUIDs validados, redirecionamentos apenas para destinos conhecidos.
RNF05	Desempenho	Respostas das APIs externas cacheadas (Data Cache do Next: 1 h na busca, 24 h no detalhe).
RNF06	Disponibilidade	*Fallback* AniList → Jikan; falha de um tipo de mídia não derruba a busca geral.
RNF07	Resiliência	Timeout de 15 s e nova tentativa em HTTP 429 ou falha de rede nas APIs externas.
RNF08	Usabilidade	Interface em português (pt-BR), tema escuro, responsiva (mobile-first).
RNF09	Portabilidade	Executável localmente (`npm run dev`), em Docker e na Vercel.
RNF10	Manutenibilidade	TypeScript estrito, ESLint, testes automatizados com Vitest e CI em todo PR.
RNF11	Degradação	Sem Supabase configurado, as páginas exibem dados de demonstração em vez de falhar.

2.3 Regras de negócio

ID	Regra
RN01	Cada usuário tem no máximo **uma** entrada por obra (`unique(user_id, media_id)`).
RN02	A nota da obra é informada de 1 a 10 e armazenada de 0 a 5 (`rating = nota / 2`, `numeric(2,1)`).
RN03	Cada usuário avalia uma campanha **uma única vez**; nova avaliação substitui a anterior (upsert).
RN04	Somente o autor ou um administrador pode editar/excluir uma campanha.
RN05	Uma obra externa é única por `(external_source, external_id)`; `external_id` inclui o tipo (`anime:20`) para evitar colisão.
RN06	Importação não sobrescreve curadoria existente (`ignoreDuplicates`).

ID	Regra
RN07	Ao criar conta, um perfil é criado automaticamente (trigger `on_auth_user_created`); o username padrão é a parte local do e-mail.
RN08	Usuários comuns não podem alterar o próprio `role` (política `profiles_update_own`).
RN09	O administrador não pode remover o próprio acesso de admin.
RN10	Listas privadas (e seus itens) só são visíveis ao dono.
RN11	Moderação de resenha remove apenas o texto; a nota é preservada.
RN12	Rótulos de status mudam conforme o tipo: mangá usa “Ler/Lendo/Lido”, demais “Assistir/Assistindo/Assistido”.
RN13	Rotas `/minha-biblioteca`, `/listas`, `/campanhas/nova` e `/admin` exigem sessão; o visitante é redirecionado para `/login?next=…`.

3. Casos de Uso

3.1 Diagrama de casos de uso

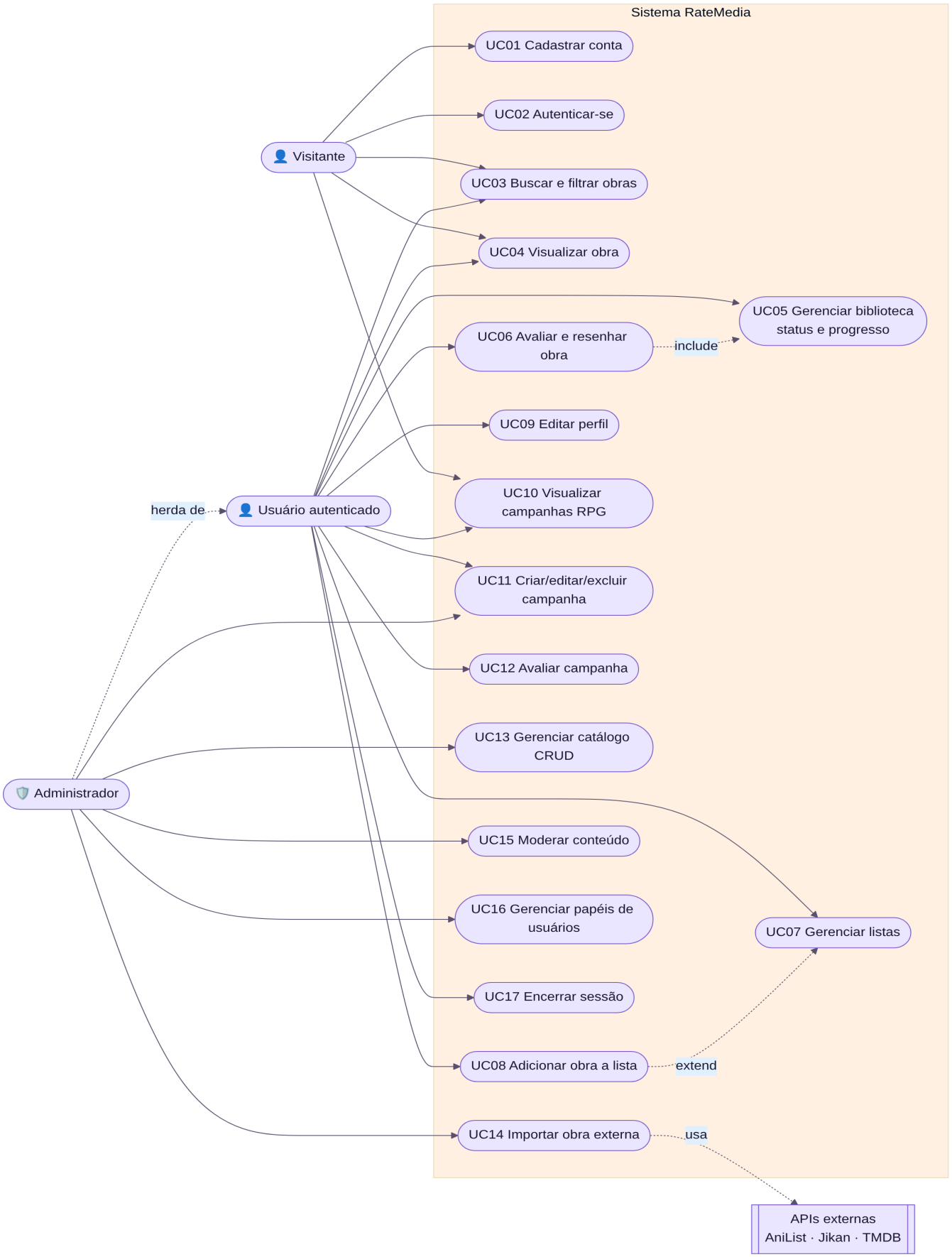


Diagrama de casos de uso

3.2 Lista de casos de uso

ID	Caso de uso	Atores	RF
UC01	Cadastrar conta	Visitante	RF01
UC02	Autenticar-se	Visitante	RF02
UC03	Buscar e filtrar obras	Visitante, Usuário	RF04–RF06
UC04	Visualizar obra	Visitante, Usuário	RF07
UC05	Gerenciar biblioteca (status e progresso)	Usuário	RF08, RF09, RF12
UC06	Avaliar e resenhar obra	Usuário	RF10, RF11
UC07	Gerenciar listas	Usuário	RF13
UC08	Adicionar obra a lista	Usuário	RF14
UC09	Editar perfil	Usuário	RF15
UC10	Visualizar campanhas de RPG	Visitante, Usuário	RF16
UC11	Criar/editar/excluir campanha	Usuário, Admin	RF17, RF18
UC12	Avaliar campanha	Usuário	RF19
UC13	Gerenciar catálogo (CRUD)	Admin	RF20, RF21
UC14	Importar obra externa	Admin, APIs externas	RF22
UC15	Moderar conteúdo	Admin	RF23
UC16	Gerenciar papéis de usuários	Admin	RF24
UC17	Encerrar sessão	Usuário	RF03

3.3 Especificações detalhadas

UC01 — Cadastrar conta

Campo	Descrição
Ator principal	Visitante
Pré-condições	Não estar autenticado.
Pós-condições	Usuário criado em `auth.users`, perfil criado em `profiles`, sessão ativa.

Fluxo principal 1. O visitante acessa `/cadastro`. 2. Informa username, e-mail e senha. 3. O sistema valida a senha (mínimo 6 caracteres). 4. O sistema cria o usuário no Supabase Auth com o username nos metadados. 5. O trigger `handle_new_user` cria o perfil. 6. O sistema redireciona para a página inicial já autenticado.

Fluxos alternativos - **A1 — Senha curta:** exibe “A senha deve ter ao menos 6 caracteres.” e permanece no formulário. - **A2 — E-mail já cadastrado / erro do provedor:** exibe a mensagem retornada pelo Supabase.

UC02 — Autenticar-se

Campo	Descrição
Ator principal	Visitante
Pré-condições	Possuir conta.
Pós-condições	Cookies de sessão gravados; navbar mostra o menu do usuário.

Fluxo principal 1. O visitante acessa `/login` (possivelmente com `?next=/rota`). 2. Informa e-mail e senha. 3. O sistema autentica via `signInWithPassword`. 4. O sistema redireciona para `next` (somente caminhos internos iniciados por `/`).

Fluxo alternativo - **A1 — Credenciais inválidas:** exibe “E-mail ou senha inválidos.”.

UC03 — Buscar e filtrar obras

Ator: Visitante ou Usuário. **Pré-condição:** nenhuma.

Fluxo principal 1. O ator digita um termo na barra de busca ou acessa `/busca`. 2. Seleciona o tipo (anime, mangá, série, filme, RPG ou todos). 3. Opcionalmente aplica faixa de nota (0–5) e gêneros. 4. Escolhe a ordenação (nota, popularidade, recentes, A–Z). 5. O sistema retorna 12 resultados por página com navegação por páginas.

Fluxo alternativo - **A1 — Nenhum resultado:** o sistema exibe estado vazio com sugestão para limpar filtros.

UC04 — Visualizar obra

1. O ator abre `/obras/{id}`.
2. O sistema exibe capa, título, subtítulo, ano, gêneros, unidades, sinopse, média de notas e críticas.
3. Críticas marcadas com spoiler ficam ocultas até o clique do leitor.
4. Se autenticado, exibe os controles de biblioteca, avaliação e “Adicionar à lista” (UC05, UC06, UC08).

UC05 — Gerenciar biblioteca

Campo	Descrição
Ator principal	Usuário autenticado
Pré-condições	Sessão ativa; obra existente.
Pós-condições	Registro em `library_entries` criado ou atualizado.

Fluxo principal 1. Na página da obra, o usuário escolhe um status. 2. Informa o progresso (episódios/capítulos). 3. O sistema executa `saveEntry`, que faz *upsert* por `(user_id, media_id)` salvando apenas os campos enviados. 4. A obra passa a aparecer em `/minha-biblioteca` no grupo do status.

Fluxo alternativo - **A1 — Sem sessão:** redireciona para `/login?next=/obras/{id}`.

UC06 — Avaliar e resenhar obra

Inclui: UC05 (a avaliação é armazenada na mesma entrada da biblioteca).

1. O usuário escolhe uma nota de 1 a 10.
2. Escreve a resenha e marca “contém spoilers”, se for o caso.
3. O sistema converte a nota (`/2`) e grava `rating`, `review` e `has_spoilers`.
4. A média da obra e a lista de críticas são revalidadas.

Alternativo: nota fora do intervalo é gravada como `null` (sem nota).

UC07 — Gerenciar listas

1. O usuário acessa `/listas` (perfil + minhas listas).
2. Cria uma lista com título (até 80 caracteres), descrição (até 300) e visibilidade.
3. Abre uma lista em `/listas/{id}` para ver ou remover itens.
4. Pode excluir a lista (somente o dono).

UC08 — Adicionar obra a lista (*extend* UC07)

1. Na página da obra, o usuário clica em “Adicionar à lista”.
2. O sistema carrega as listas do usuário indicando em quais a obra já está.
3. O usuário marca/desmarca listas; o sistema insere/remove `list_items` na última posição.
4. **Alternativo:** o usuário cria uma nova lista que já nasce com a obra.
5. **Exceção:** obra já presente (erro `23505`) é tratada como sucesso.

UC09 — Editar perfil

1. Em `/listas`, o usuário edita bio (300), localização (80), URL do avatar e URL do banner.
2. URLs que não começam com `http(s)://` são descartadas.
3. O sistema atualiza `profiles` do próprio usuário.

UC10 — Visualizar campanhas

1. O ator acessa `/campanhas`.
2. O sistema lista campanhas com sistema, status, nota média e número de avaliações.
3. Ao abrir `/campanhas/{id}`, vê sinopse, regras e avaliações.

UC11 — Criar/editar/excluir campanha

Campo	Descrição
Atores	Usuário (autor), Administrador
Pré-condições	Sessão ativa; para editar/excluir, ser autor ou admin.

Fluxo principal (criar) 1. O usuário acessa `/campanhas/nova`. 2. Preenche título (150), sistema (80), sinopse (4000), regras (4000), status e URL da capa. 3. O sistema valida campos obrigatórios (título, sistema, sinopse) e grava com `author_id` do usuário. 4. Redireciona para `/campanhas/{id}`.

Fluxo alternativo (editar/excluir) 1. Autor ou admin acessa `/campanhas/{id}/editar` ou o painel `/admin/campanhas`. 2. O sistema executa `authorize()` (getViewer + `canManageCampaign`). 3. Atualiza ou exclui; exclusão redireciona apenas para `/campanhas` ou `/admin/campanhas`.

Exceção: usuário sem permissão — a operação é ignorada (e bloqueada também pela RLS).

UC12 — Avaliar campanha

1. Na página da campanha, o usuário define nota (0–5) e comentário.
2. O sistema faz *upsert* por `(campaign_id, user_id)`.
3. A média da campanha é recalculada na próxima renderização.

UC13 — Gerenciar catálogo

Ator: Administrador. **Pré-condição:** `role = 'admin'`.

1. O admin acessa `/admin/catalogo` e vê a tabela de obras.
2. Cria (`/admin/catalogo/nova`) ou edita (`/admin/catalogo/{id}`) obras: tipo, título, subtítulo, sinopse, capa, ano, unidades, até 8 gêneros e status editorial.
3. Exclui obras (exclusão em cascata remove entradas e itens de lista).

UC14 — Importar obra externa

Atores: Administrador, APIs externas.

1. O admin acessa `/admin/catalogo/importar` e pesquisa por título e tipo.
2. O sistema consulta AniList (anime/mangá, com *fallback* para Jikan) ou TMDB (filme/série).
3. O admin clica em “Importar”; o formulário envia apenas tipo, fonte e id externo.
4. O servidor busca os dados completos na API (o cliente não injeta conteúdo) e grava com `ignoreDuplicates`.
5. O sistema exibe confirmação (`?ok=1`).

Exceções: `erro=api` (API indisponível), `erro=nao-encontrada`, `erro=db`.

UC15 — Moderar conteúdo

1. O admin acessa `/admin/moderacao`.
2. Pode limpar o texto de uma resenha (nota preservada) ou excluir uma avaliação de campanha.

UC16 — Gerenciar papéis

1. O admin acessa `/admin/usuarios`.
2. Altera o papel de outro usuário para `admin` ou `user`.
3. **Restrição:** o admin não pode alterar o próprio papel.

UC17 — Encerrar sessão

1. O usuário abre o menu do avatar e clica em “Sair”.
2. O sistema invalida a sessão e redireciona para `/`.

4. Histórias de Usuário

Formato: **Como** <ator>, **quero** <ação>, **para** <benefício>. Estimativas em *story points* (Fibonacci). Critérios de aceitação no formato *Dado / Quando / Então*.

Épico E1 — Conta e acesso

US01 — Criar conta

Como visitante, **quero** criar uma conta com username, e-mail e senha, **para** salvar meu histórico de mídias. - **Pontos:** 3 · **Prioridade:** Alta · **UC:** UC01 - **Critérios de aceitação** - Dado que informei uma senha com menos de 6 caracteres, quando envio o formulário, então vejo a mensagem de erro e a conta não é criada. - Dado dados válidos, quando envio, então sou autenticado e redirecionado para a home. - Dado que me cadastrei, então um perfil com meu username é criado automaticamente.

US02 — Entrar na conta

Como usuário, **quero** fazer login com e-mail e senha, **para** acessar minha biblioteca. - **Pontos:** 2 · **Prioridade:** Alta · **UC:** UC02 - **Critérios** - Dado credenciais inválidas, então vejo “E-mail ou senha inválidos.”. - Dado que tentei abrir uma rota protegida, quando faço login, então volto para essa rota.

US03 — Sair da conta

Como usuário, **quero** encerrar minha sessão, **para** proteger minha conta em computadores compartilhados. - **Pontos:** 1 · **Prioridade:** Alta · **UC:** UC17

Épico E2 — Descoberta

US04 — Buscar obras

Como visitante, **quero** buscar obras por nome e tipo, **para** encontrar rapidamente o que procuro. - **Pontos:** 5 · **Prioridade:** Alta · **UC:** UC03 - **Critérios** - Quando digito um termo, então vejo animes, mangás, séries, filmes e campanhas correspondentes. - Os resultados são paginados com 12 itens por página.

US05 — Filtrar e ordenar

Como usuário, **quero** filtrar por nota e gênero e ordenar por popularidade, nota, recência ou A–Z, **para** descobrir obras relevantes. - **Pontos:** 5 · **Prioridade:** Média · **UC:** UC03 - **Critérios** - Dado um filtro de nota mínima, então obras sem nota não aparecem. - Dado vários gêneros, então aparece qualquer obra com pelo menos um deles.

US06 — Ver detalhes da obra

Como visitante, **quero** ver sinopse, metadados, média e críticas de uma obra, **para** decidir se vou assisti-la/lê-la. - **Pontos:** 3 · **Prioridade:** Alta · **UC:** UC04 - **Critérios:** críticas com spoiler ficam ocultas até eu escolher revelá-las.

Épico E3 — Biblioteca e avaliações

US07 — Marcar status

Como usuário, **quero** marcar uma obra como “Quero assistir”, “Assistindo”, “Assistido” ou “Abandonado”, **para** organizar meu consumo. - **Pontos:** 3 · **Prioridade:** Alta · **UC:** UC05 - **Critérios** - Para mangás os rótulos são “Quero ler / Lendo / Lido”. - Só existe uma entrada por obra por usuário.

US08 — Registrar progresso

Como usuário, **quero** registrar o episódio/capítulo em que parei, **para** retomar de onde parei. - **Pontos:** 2 · **Prioridade:** Média · **UC:** UC05 - **Critérios:** valores negativos ou inválidos viram 0.

US09 — Dar nota

Como usuário, **quero** dar uma nota de 1 a 10, **para** registrar minha opinião e compor a média da comunidade. - **Pontos:** 2 · **Prioridade:** Alta · **UC:** UC06 - **Critérios:** a média exibida na obra considera todas as notas registradas.

US10 — Escrever resenha

Como usuário, **quero** escrever uma resenha e marcar se contém spoilers, **para** compartilhar minha análise sem estragar a experiência de outros. - **Pontos:** 3 · **Prioridade:** Alta · **UC:** UC06

US11 — Ver minha biblioteca

Como usuário, **quero** ver todas as obras da minha biblioteca agrupadas por status, **para** ter uma visão geral do meu consumo. - **Pontos:** 3 · **Prioridade:** Alta · **UC:** UC05

Épico E4 — Listas e perfil

US12 — Criar listas

Como usuário, **quero** criar listas públicas ou privadas, **para** organizar coleções temáticas (“Top 10 isekai”). - **Pontos:** 3 · **Prioridade:** Média · **UC:** UC07 - **Critérios:** listas privadas não aparecem para outros usuários.

US13 — Adicionar obra à lista

Como usuário, **quero** adicionar uma obra a uma ou mais listas diretamente da página da obra, **para** não perder tempo navegando. - **Pontos:** 5 · **Prioridade:** Média · **UC:** UC08 - **Critérios** - Vejo em quais listas a obra já está. - Posso criar uma nova lista que já contém a obra. - Adicionar a mesma obra duas vezes não gera erro nem duplicata.

US14 — Personalizar perfil

Como usuário, **quero** editar bio, localização, avatar e banner, **para** expressar minha identidade na comunidade. - **Pontos:** 2 · **Prioridade:** Baixa · **UC:** UC09

Épico E5 — Campanhas de RPG

US15 — Explorar campanhas

Como jogador, **quero** ver campanhas com sistema, status e nota, **para** encontrar uma mesa que combine comigo. - **Pontos:** 3 · **Prioridade:** Alta · **UC:** UC10

US16 — Publicar campanha

Como mestre, **quero** publicar minha campanha com sistema, sinopse, regras e status, **para** recrutar jogadores. - **Pontos:** 5 · **Prioridade:** Alta · **UC:** UC11 - **Critérios:** título, sistema e sinopse são obrigatórios; a capa só aceita URL `http(s)`.

US17 — Gerenciar minha campanha

Como mestre, **quero** editar ou excluir minha campanha, **para** mantê-la atualizada. - **Pontos:** 3 · **Prioridade:** Alta · **UC:** UC11 - **Critérios:** outro usuário não consegue editar nem excluir a campanha.

US18 — Avaliar campanha

Como jogador, **quero** avaliar uma campanha com nota e comentário, **para** ajudar outros jogadores. - **Pontos:** 2 · **Prioridade:** Alta · **UC:** UC12 - **Critérios:** uma nova avaliação minha substitui a anterior.

Épico E6 — Administração

US19 — Painel de visão geral

Como administrador, **quero** ver métricas do sistema (obras indexadas, rascunhos em revisão, usuários, avaliações), **para** acompanhar a plataforma. - **Pontos:** 3 · **Prioridade:** Média · **UC:** UC13

US20 — Curar catálogo

Como administrador, **quero** criar, editar e excluir obras com status editorial, **para** manter o catálogo correto. - **Pontos:** 5 · **Prioridade:** Alta · **UC:** UC13

US21 — Importar de APIs externas

Como administrador, **quero** importar obras do AniList, Jikan e TMDB, **para** popular o catálogo sem digitação manual. - **Pontos:** 8 · **Prioridade:** Alta · **UC:** UC14 - **Critérios** - Se o AniList estiver fora do ar, a busca usa o Jikan. - Importar a mesma obra novamente não sobrescreve minhas alterações. - Anime e mangá com o mesmo id externo não colidem.

US22 — Moderar conteúdo

Como administrador, **quero** remover resenhas e avaliações abusivas, **para** manter a comunidade saudável. - **Pontos:** 3 · **Prioridade:** Alta · **UC:** UC15 - **Critérios:** limpar uma resenha preserva a nota do usuário.

US23 — Gerenciar papéis

Como administrador, **quero** promover ou rebaixar usuários, **para** distribuir a moderação. - **Pontos:** 2 · **Prioridade:** Média · **UC:** UC16 - **Critérios:** não consigo remover meu próprio acesso de admin.

Épico E7 — DevOps e documentação

US24 — Pipeline de integração contínua

Como desenvolvedor, **quero** que lint, typecheck, testes e build rodem em cada PR, **para** evitar regressões. - **Pontos:** 3 · **Prioridade:** Alta

US25 — Deploy contínuo

Como equipe, **quero** que cada PR gere um preview e a `main` publique em produção na Vercel, **para** validar mudanças rapidamente. - **Pontos:** 2 · **Prioridade:** Alta

US26 — Documentação acessível

Como avaliador/novo integrante, **quero** acessar toda a documentação em `/docs` nos formatos Markdown, Word, PDF e diagramas, **para** entender o projeto sem precisar ler o código. - **Pontos:** 5 · **Prioridade:** Média

Resumo do backlog

Épico	Histórias	Pontos
E1 Conta e acesso	US01–US03	6
E2 Descoberta	US04–US06	13
E3 Biblioteca e avaliações	US07–US11	13
E4 Listas e perfil	US12–US14	10
E5 Campanhas de RPG	US15–US18	13
E6 Administração	US19–US23	21
E7 DevOps e documentação	US24–US26	10
Total	**26**	**86**

5. Arquitetura

5.1 Visão geral

O RateMedia é um **monólito Next.js** (front-end e back-end no mesmo projeto) que usa o **Supabase** como *Backend-as-a-Service* (autenticação + PostgreSQL). As páginas são **Server Components** que consultam o banco diretamente; mutações são feitas por **Server Actions** chamadas pelos formulários. A segurança é aplicada em duas camadas: validação no servidor e **RLS** no banco.

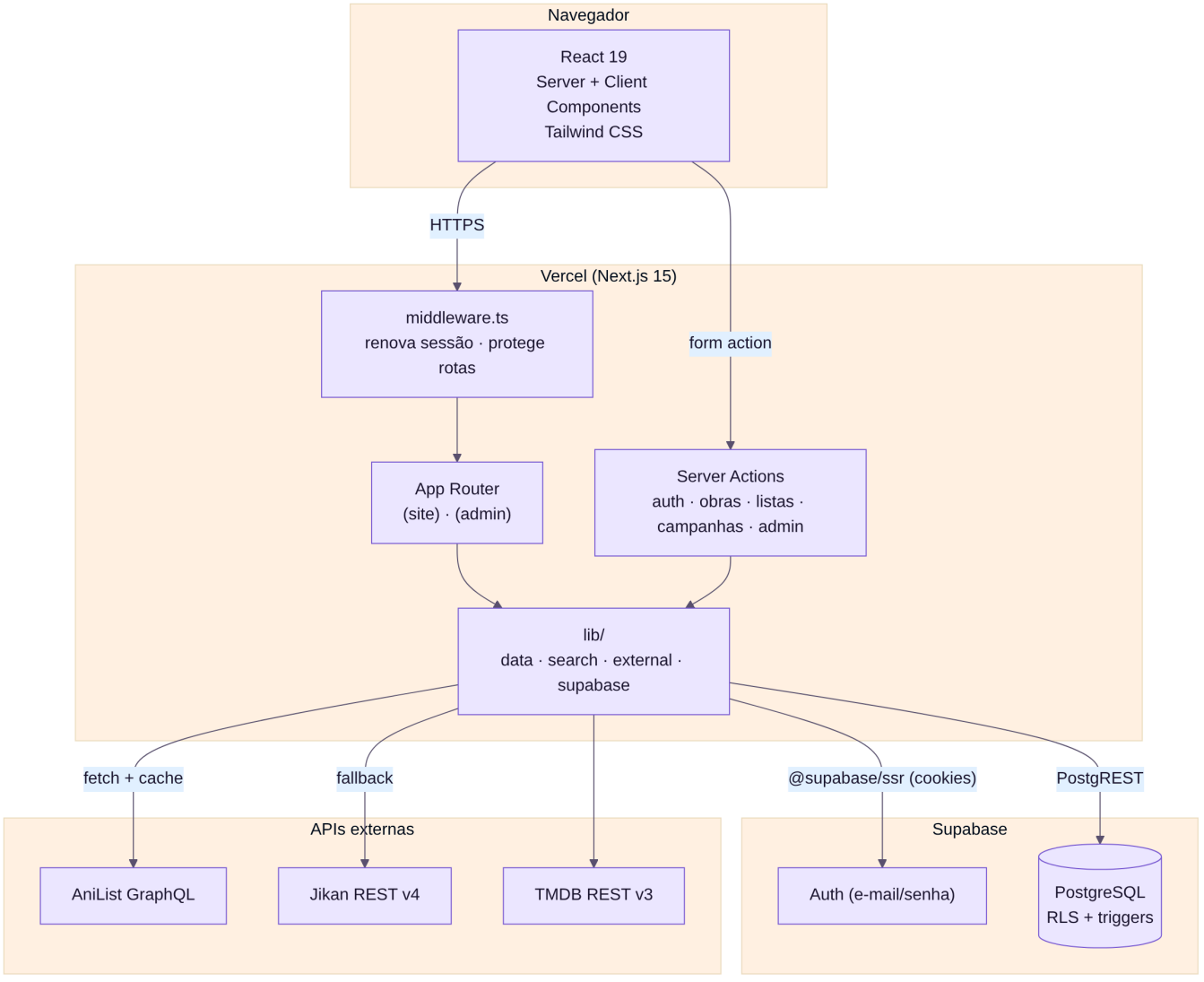


Diagrama de arquitetura

5.2 Camadas

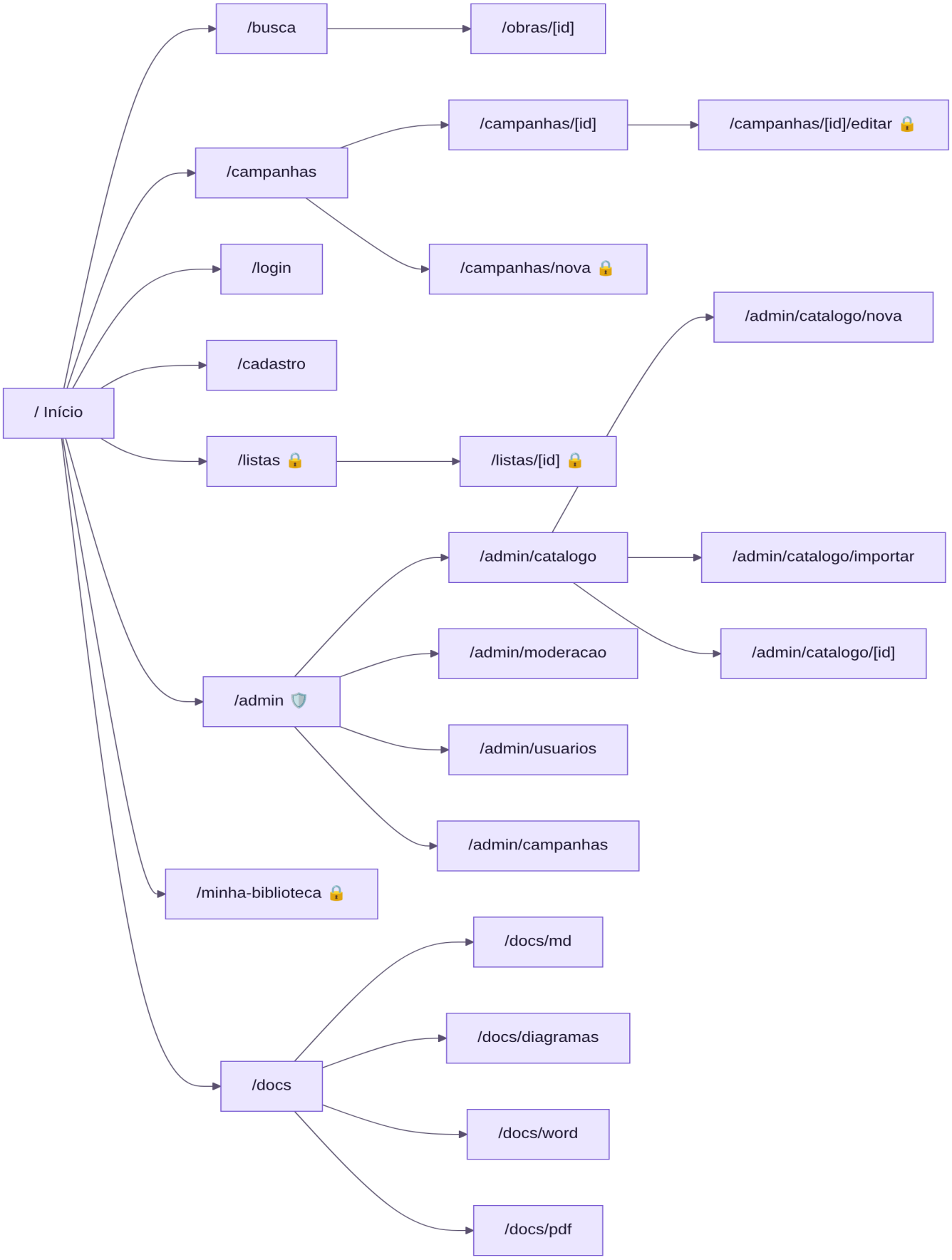
Camada	Local	Responsabilidade
Apresentação	`src/app/**/page.tsx`, `src/components/**`	Renderização (RSC + componentes cliente), Tailwind.

Camada	Local	Responsabilidade
Roteamento/Proteção	`src/middleware.ts`	Renova a sessão Supabase em cada requisição e redireciona visitantes de rotas protegidas.
Aplicação	`src/app/**/actions.ts`, `src/lib/actions/*`	Server Actions: validação, autorização, mutações e `revalidatePath`.
Domínio/Utilitários	`src/lib/data.ts`, `search.ts`, `format.ts`, `constants.ts`	Médias, estatísticas, filtros, ordenação, paginação, rótulos.
Integração	`src/lib/external/*`	Clientes AniList/Jikan/TMDB, normalização (`ExternalMedia`), retry/timeout.
Dados	`src/lib/supabase/*`, `supabase/*.sql`	Clientes Supabase (server/browser), schema, migrações e políticas RLS.

5.3 Estrutura de pastas

```
src/
  app/
    (site)/                 site público (Navbar + Footer)
      (auth)/               login, cadastro e actions de autenticação
      (app)/busca/          busca avançada
      (app)/listas/         perfil + listas (e /listas/[id])
      (app)/minha-biblioteca/
      obras/[id]/           detalhe da obra
      campanhas/            listagem, criação, edição e detalhe
      docs/                 central de documentação (md, diagramas, word, pdf)
    (admin)/admin/          painel administrativo (layout com sidebar)
  components/               UI reutilizável (home, busca, obra, perfil, admin, docs)
  lib/                      supabase, external, data, search, format, constants
  middleware.ts
  types/index.ts            tipos de domínio
supabase/
  schema.sql                tabelas, enums, triggers, RLS
  migrations/               002 (gêneros, perfil, admin) e 003 (correções)
public/docs/                arquivos da documentação (md, diagramas, word, pdf)
```

5.4 Mapa de navegação



Mapa de navegação

🔒 = exige sessão (middleware) · 🛡️ = exige `role = admin` (`requireAdmin()` + RLS).

5.5 Decisões arquiteturais (ADR resumidas)

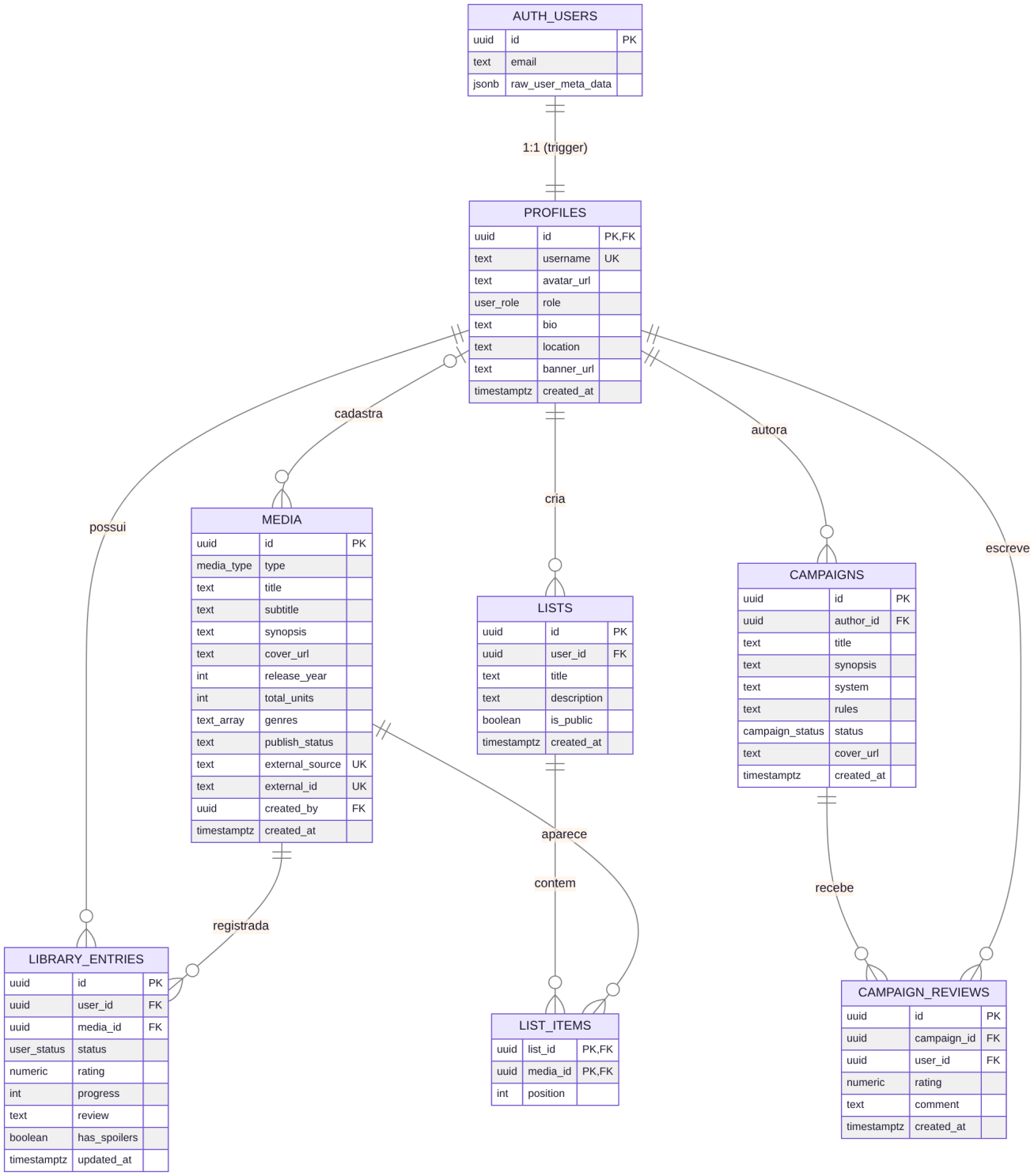
#	Decisão	Motivo	Consequência
ADR-01	Next.js App Router com Server Actions	Um único projeto para UI e API, sem camada REST própria.	Menos código; acoplamento ao Next.
ADR-02	Supabase (Auth + Postgres)	Autenticação pronta, Postgres gerenciado e plano gratuito.	Dependência de serviço externo.
ADR-03	Segurança por RLS	A regra de acesso vive no banco e vale para qualquer cliente.	Políticas precisam ser testadas e versionadas.
ADR-04	Importação feita no servidor	O cliente envia só `type/source/id`; impede injeção de dados.	Uma chamada extra à API na importação.
ADR-05	AniList como fonte principal e Jikan como *fallback*	Disponibilidade variável das APIs.	Dois normalizadores para manter.
ADR-06	Dados de demonstração sem Supabase	Permite avaliar a UI sem credenciais (Docker/preview).	Páginas precisam tratar os dois modos.
ADR-07	`output: "standalone"`	Imagem Docker pequena.	Nenhuma no deploy Vercel.

5.6 Fluxo de autenticação e sessão

1. `@supabase/ssr` guarda o token em cookies HTTP.
2. O `middleware.ts` cria um cliente com os cookies da requisição, chama `auth.getUser()` (renovando o token se preciso) e reescreve os cookies na resposta.
3. Se a rota estiver em `PROTECTED` e não houver usuário, redireciona para `/login?next=<rota>`.
4. Server Components e Server Actions usam `createClient()` de `lib/supabase/server.ts`, que lê os mesmos cookies.

6. Modelo de Dados

6.1 Diagrama Entidade-Relacionamento (DER)



DER

6.2 Tipos enumerados

Enum	Valores
`media_type`	`anime`, `manga`, `series`, `movie`
`user_status`	`planned`, `in_progress`, `completed`, `dropped`
`campaign_status`	`recruiting`, `ongoing`, `finished`, `paused`
`user_role`	`user`, `admin`
`publish_status` (check)	`published`, `draft`, `review`

6.3 Dicionário de dados

profiles

Coluna	Tipo	Restrições	Descrição
id	uuid	PK, FK → auth.users (cascade)	Mesmo id do usuário de autenticação.
username	text	unique, not null	Nome público.
avatar_url	text		URL da foto.
role	user_role	not null, default `user`	Papel de acesso.
bio	text		Biografia (≤ 300).
location	text		Localização (≤ 80).
banner_url	text		URL do banner.
created_at	timestamptz	default now()	Data de criação.

media

Coluna	Tipo	Restrições	Descrição
id	uuid	PK	Identificador.
type	media_type	not null, índice	Tipo da obra.
title	text	not null, índice GIN full-text	Título.
subtitle	text		Estúdio, autor ou emissora.
synopsis	text		Sinopse (≤ 4000).
cover_url	text		Capa.
release_year	int		Ano de lançamento.
total_units	int		Episódios/capítulos/1.
genres	text[]	not null, default `{}`, índice GIN	Gêneros (≤ 8).
publish_status	text	check, default `published`	Status editorial.
external_source	text	unique (com external_id)	`anilist`, `jikan`, `tmdb` ou `manual`.

Coluna	Tipo	Restrições	Descrição
external_id	text	unique (com external_source)	`tipo:id` (ex.: `anime:20`).
created_by	uuid	FK → profiles	Admin que cadastrou.
created_at	timestamptz	default now()	

library_entries

Coluna	Tipo	Restrições	Descrição
id	uuid	PK	
user_id	uuid	FK → profiles (cascade), unique com media_id	Dono.
media_id	uuid	FK → media (cascade)	Obra.
status	user_status	default `planned`	Situação.
rating	numeric(2,1)	check 0–5	Nota.
progress	int	default 0	Unidades consumidas.
review	text		Resenha.
has_spoilers	boolean	default false	Marca de spoiler.
updated_at	timestamptz	default now()	Última alteração.

lists / list_items

Tabela.Coluna	Tipo	Restrições	Descrição
lists.id	uuid	PK	
lists.user_id	uuid	FK → profiles (cascade)	Dono.
lists.title	text	not null	Título (≤ 80).
lists.description	text		Descrição (≤ 300).
lists.is_public	boolean	default true	Visibilidade.
list_items.list_id	uuid	PK, FK → lists (cascade)	
list_items.media_id	uuid	PK, FK → media (cascade)	
list_items.position	int	default 0	Ordem na lista.

campaigns / campaign_reviews

Tabela.Coluna	Tipo	Restrições	Descrição
campaigns.id	uuid	PK	
campaigns.author_id	uuid	FK → profiles (cascade)	Mestre.
campaigns.title	text	not null	(≤ 150).
campaigns.synopsis	text	not null	(≤ 4000).

Tabela.Coluna	Tipo	Restrições	Descrição
campaigns.system	text	not null	Ex.: D&D 5e, Tormenta20.
campaigns.rules	text		Regras da mesa.
campaigns.status	campaign_status	default `recruiting`	
campaigns.cover_url	text		
campaign_reviews.id	uuid	PK	
campaign_reviews.campaign_id	uuid	FK → campaigns (cascade), unique com user_id	
campaign_reviews.user_id	uuid	FK → profiles (cascade)	
campaign_reviews.rating	numeric(2,1)	not null, check 0–5	
campaign_reviews.comment	text		

6.4 Funções e triggers

Objeto	Tipo	Descrição
`is_admin()`	função SQL `security definer`	Retorna se `auth.uid()` tem `role = 'admin'`. Usada nas políticas.
`handle_new_user()`	função PL/pgSQL	Cria o perfil com o username dos metadados ou a parte local do e-mail.
`on_auth_user_created`	trigger `after insert on auth.users`	Executa `handle_new_user()`.

6.5 Políticas de Row Level Security

Tabela	Leitura	Escrita
profiles	todos	o próprio (sem trocar `role`); admin atualiza qualquer um
media	todos	somente admin
library_entries	todos	o dono; admin pode atualizar (moderação)
lists	públicas ou do dono	o dono
list_items	se a lista for pública ou do dono	o dono da lista
campaigns	todos	autor ou admin
campaign_reviews	todos	autor da avaliação ou admin

6.6 Migrações

Arquivo	Conteúdo
`supabase/schema.sql`	Schema inicial: enums, tabelas, índices, `is_admin`, trigger e RLS.
`migrations/002_telas_busca_perfil_admin.sql`	`genres`, `subtitle`, `publish_status`; `bio`, `location`, `banner_url`; política `profiles_admin_update`.
`migrations/003_fix_has_spoilers_e_colunas.sql`	Garante `has_spoilers`, `review`, `progress`, `is_public` e recarrega o schema do PostgREST.

7. Diagrama de Classes

O diagrama abaixo representa o **modelo de domínio** (interfaces TypeScript em `src/types/index.ts`, que espelham as tabelas) e a **camada de integração** com APIs externas (`src/lib/external`).

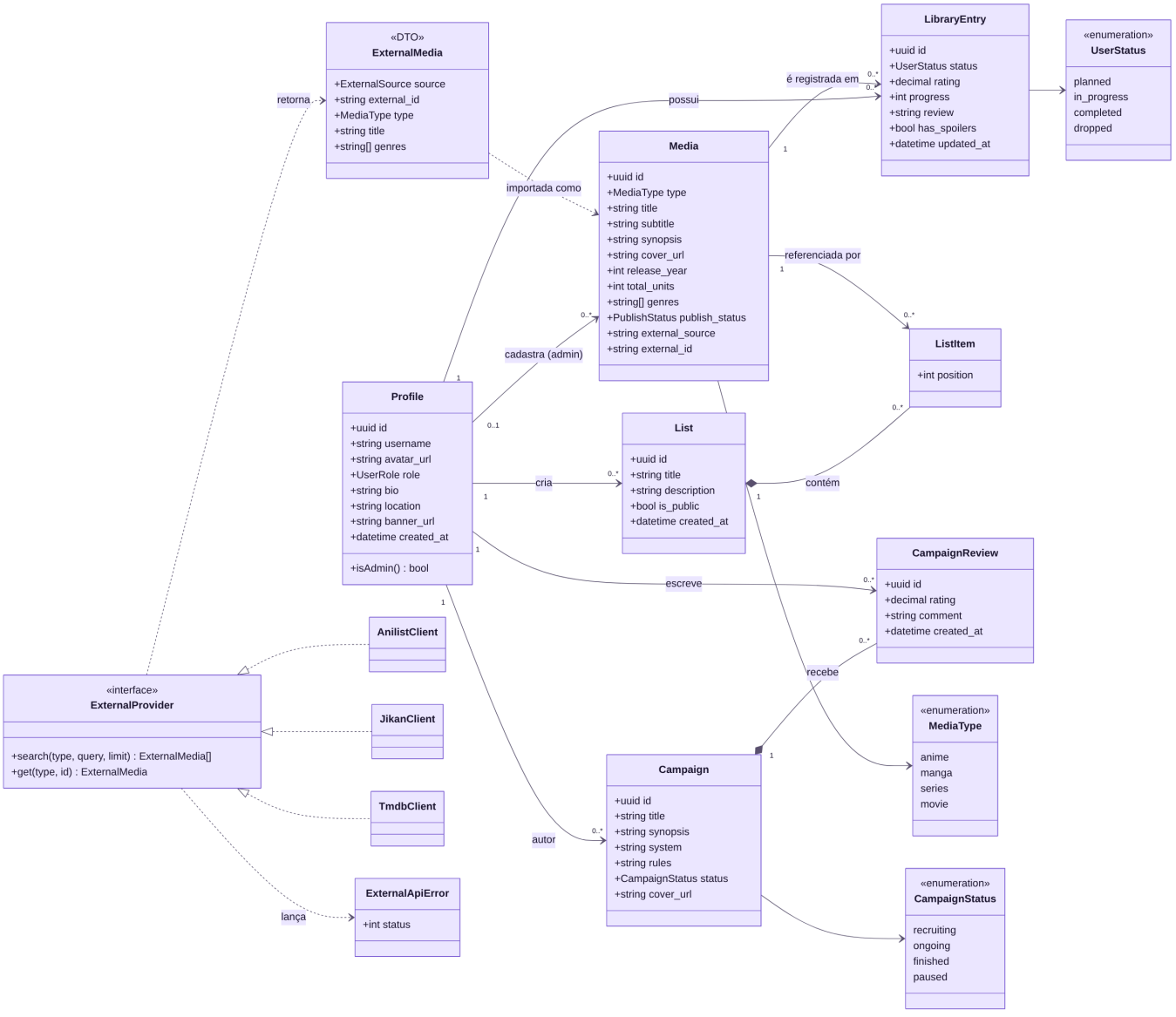


Diagrama de classes

7.1 Classes de domínio

Classe	Arquivo	Responsabilidade
`Profile`	`types/index.ts`	Usuário público; papel `user` / `admin`.
`Media`	`types/index.ts`	Obra do catálogo.
`LibraryEntry`	`types/index.ts`	Relação usuário × obra com status, progresso, nota e resenha.
`List` / `ListItem`	tabelas `lists`, `list_items`	Coleções ordenadas de obras.

Classe	Arquivo	Responsabilidade
`Campaign`	`types/index.ts`	Campanha de RPG.
`CampaignReview`	`types/index.ts`	Avaliação de campanha.

7.2 Integração externa

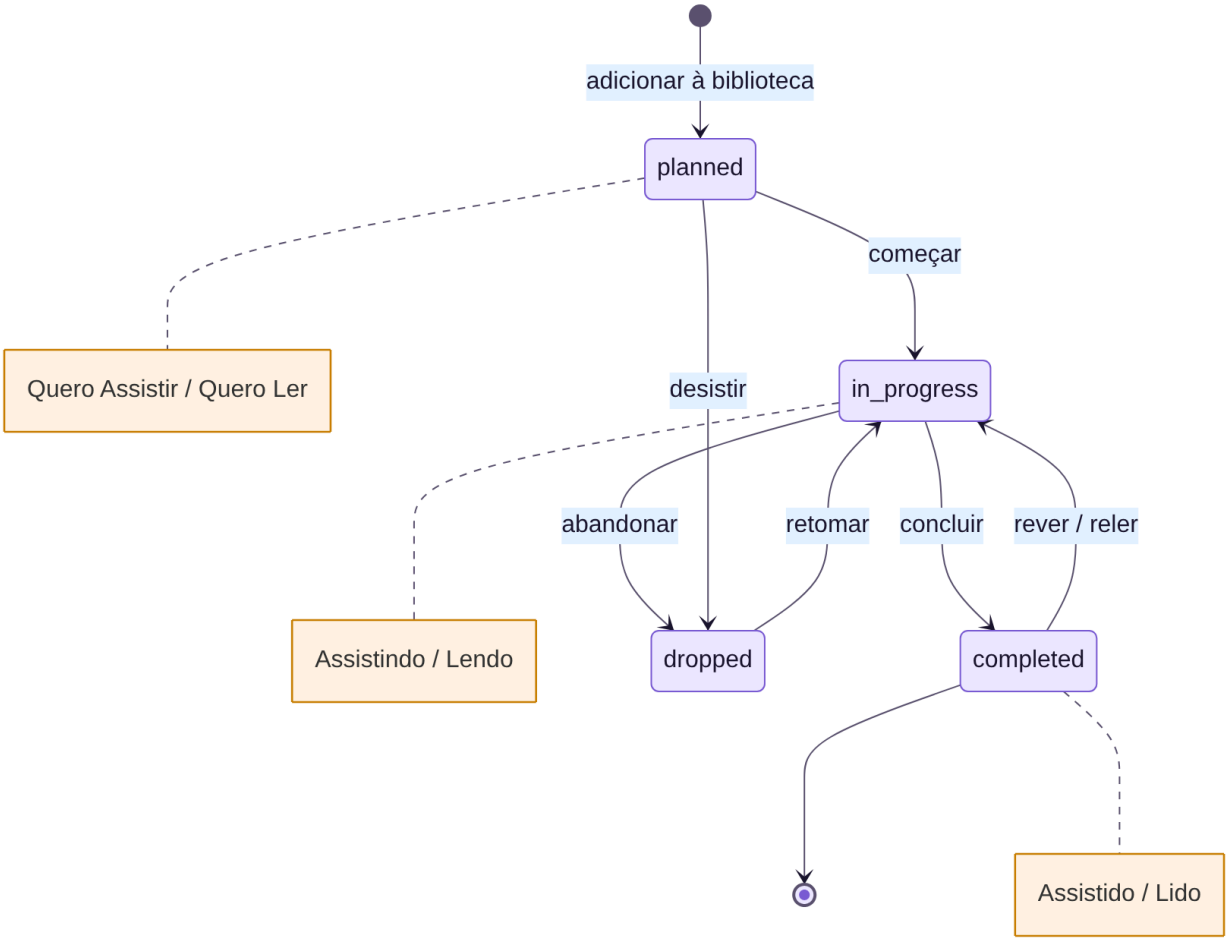
Elemento	Arquivo	Descrição
`ExternalMedia`	`external/types.ts`	DTO normalizado de qualquer fonte.
`ExternalProvider` (conceitual)	`external/{anilist,jikan,tmdb}.ts`	Cada módulo expõe `search*` e `get*`.
`ExternalApiError`	`external/types.ts`	Erro de rede/HTTP; dispara o *fallback*.
`fetchJson`	`external/http.ts`	GET/POST com timeout, cache e retry.
`searchExternal`, `searchAllExternal`, `fetchExternal`	`external/index.ts`	Fachada que roteia por tipo e fonte.
`SOURCES_BY_TYPE`, `storageId`	`external/types.ts`	Fontes permitidas por tipo e chave única `tipo:id`.

7.3 Serviços e utilitários

Função	Arquivo	Descrição
`average`, `mediaStats`, `campaignStats`	`lib/data.ts`	Médias e contagem de avaliações.
`getViewer`, `requireAdmin`, `canManageCampaign`	`lib/data.ts`	Autorização no servidor.
`filterItems`, `sortItems`, `paginate`, `buildQuery`	`lib/search.ts`	Busca avançada.
`statusLabel`	`lib/constants.ts`	Rótulos por tipo (ler × assistir).
`timeAgo`	`lib/format.ts`	Datas relativas em pt-BR.

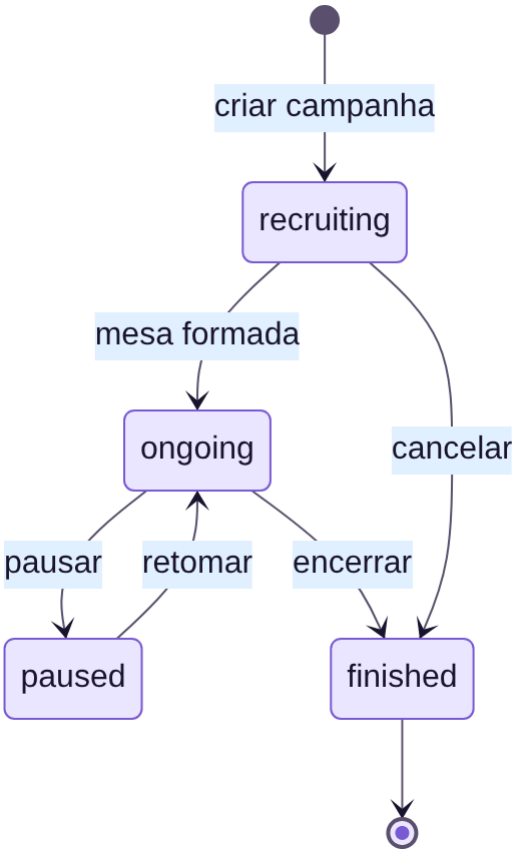
7.4 Diagramas de estado

Status de uma obra na biblioteca



Estados da biblioteca

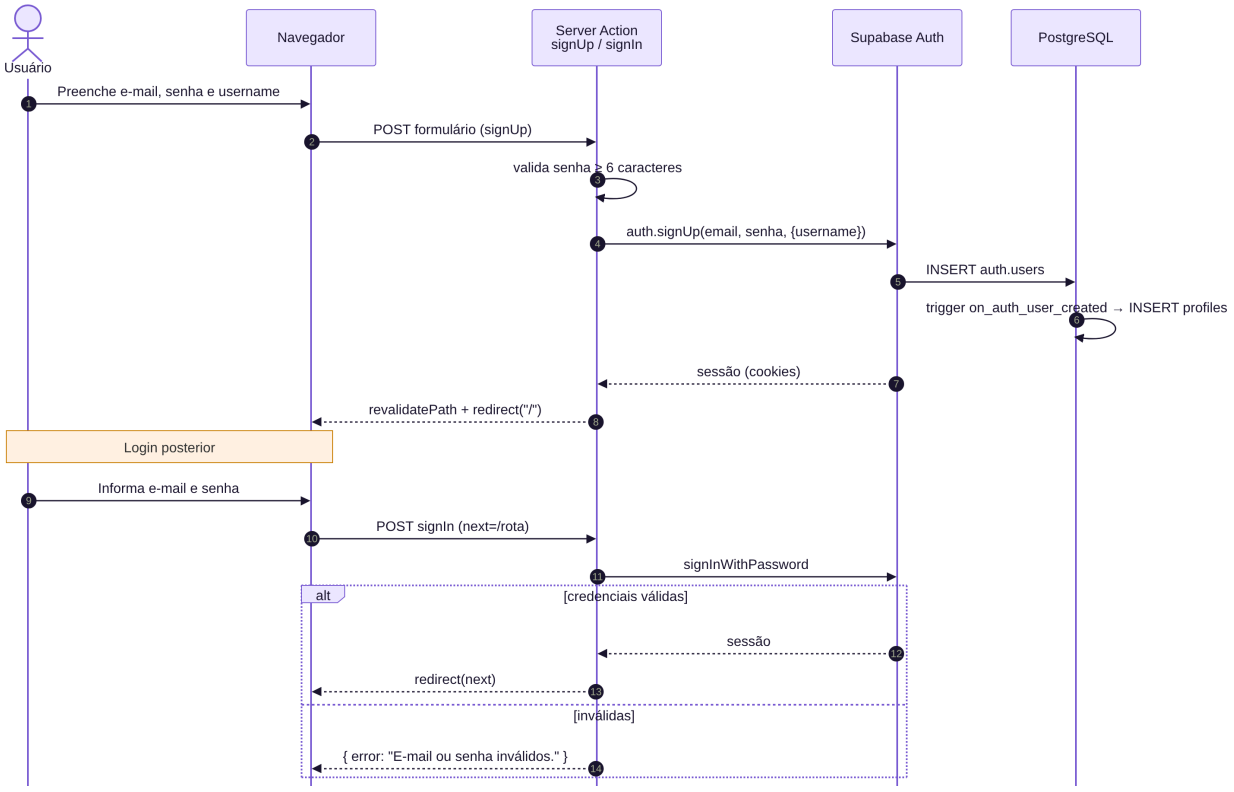
Status de uma campanha



Estados da campanha

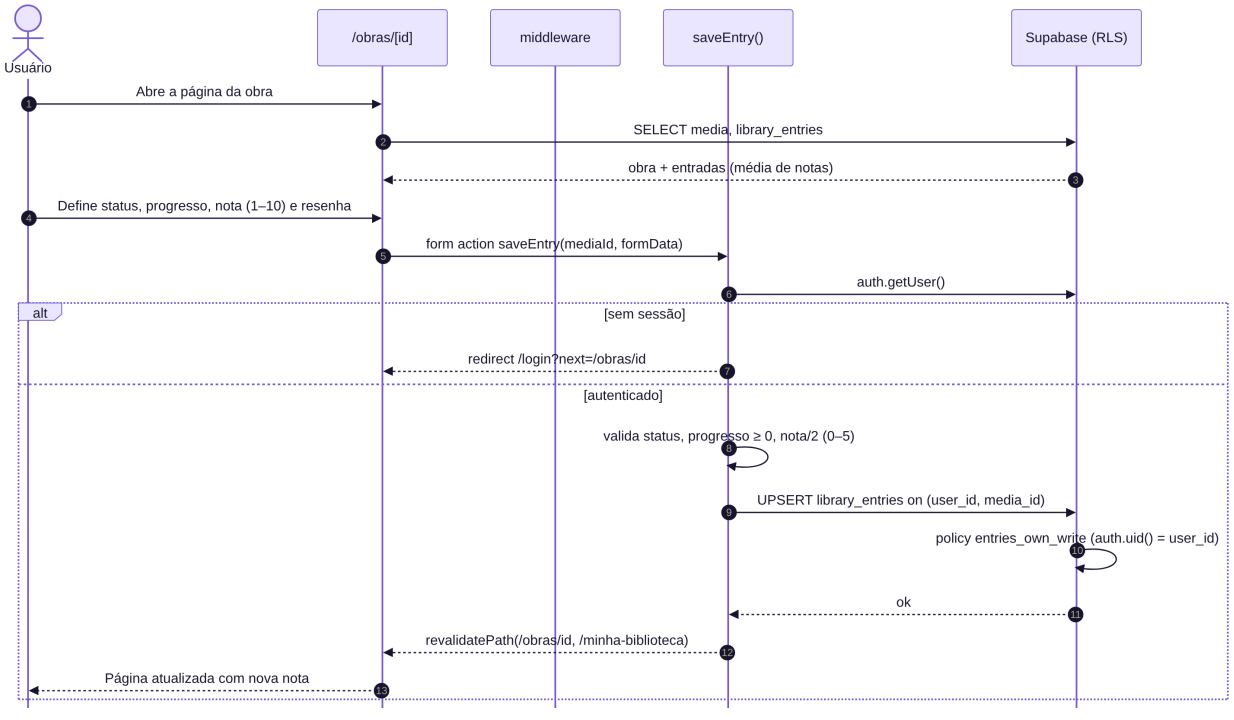
8. Diagramas de Sequência

8.1 Cadastro e login (UC01, UC02)



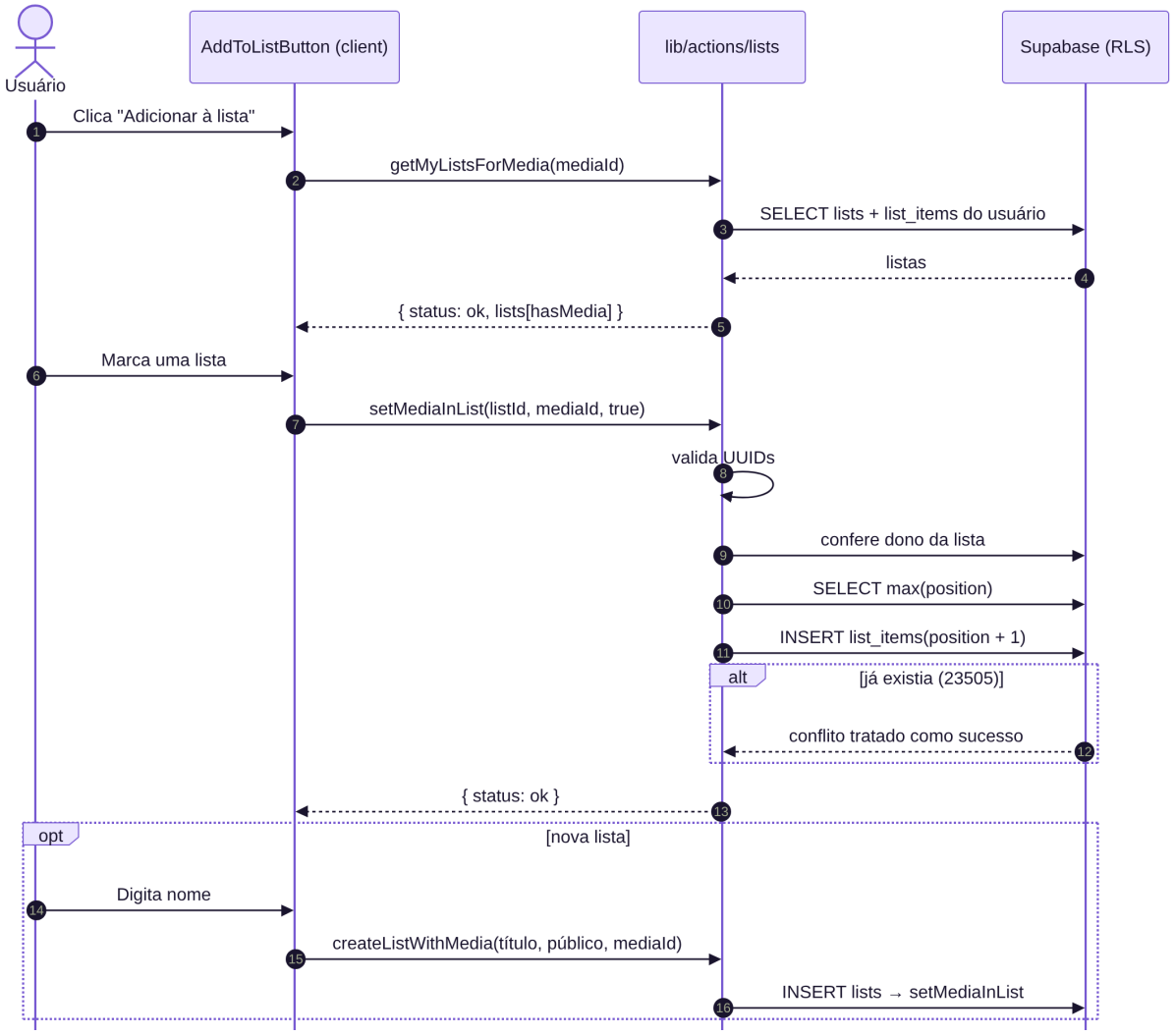
Sequência — cadastro e login

8.2 Avaliar e resenhar obra (UC05, UC06)



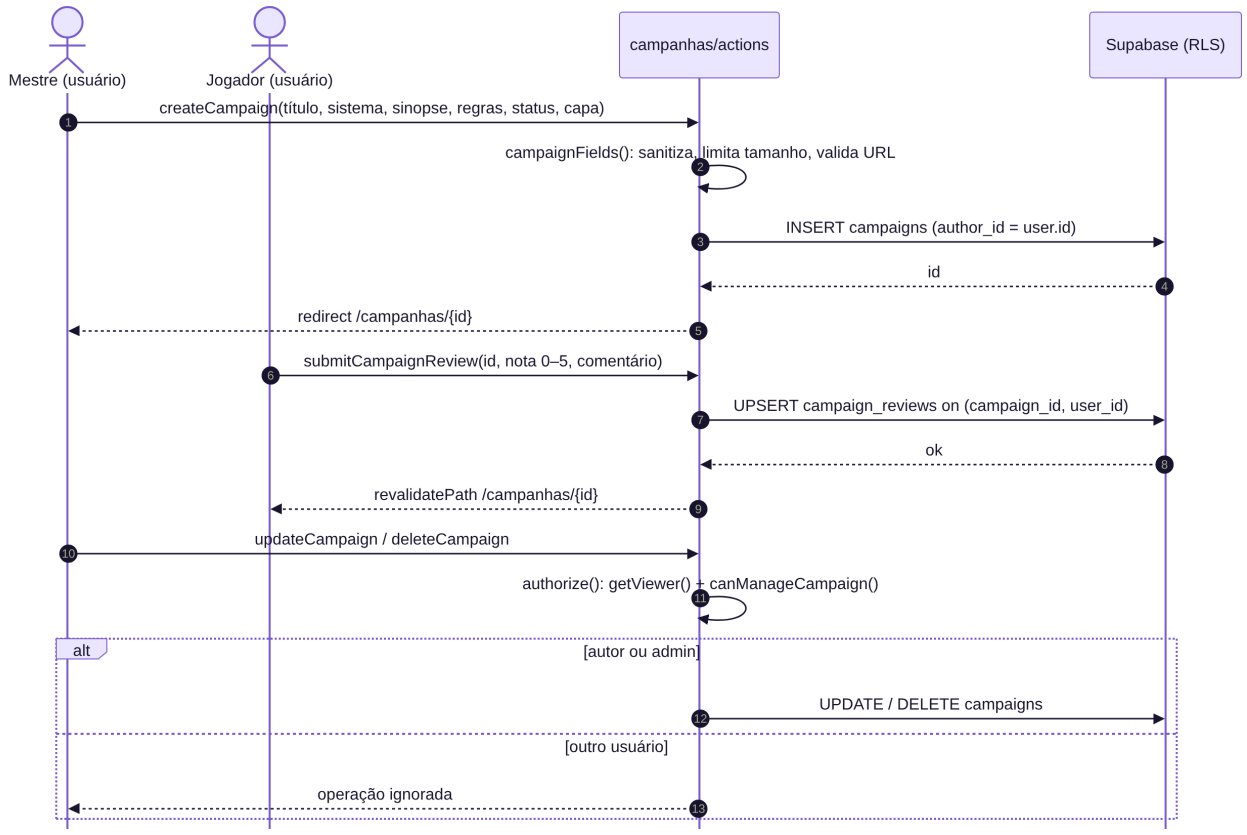
Sequência — avaliar obra

8.3 Adicionar obra a uma lista (UC08)



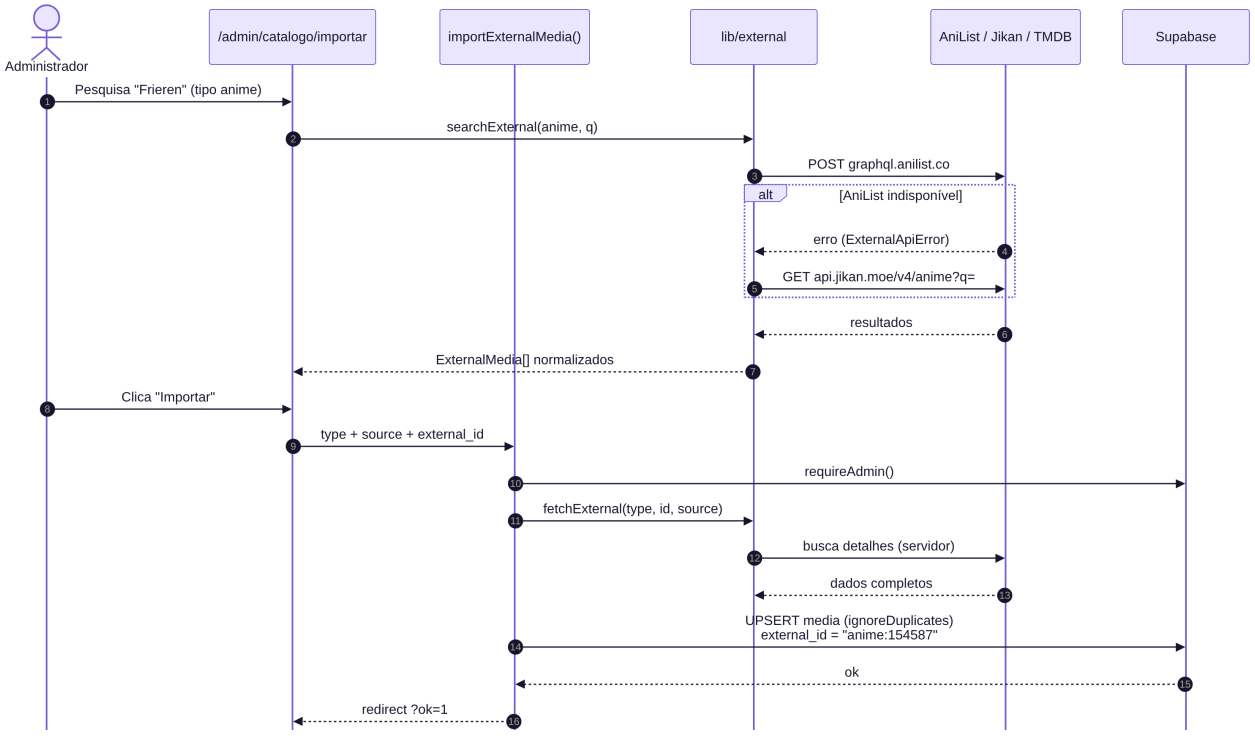
Sequência — adicionar à lista

8.4 Criar, avaliar e gerenciar campanha (UC11, UC12)



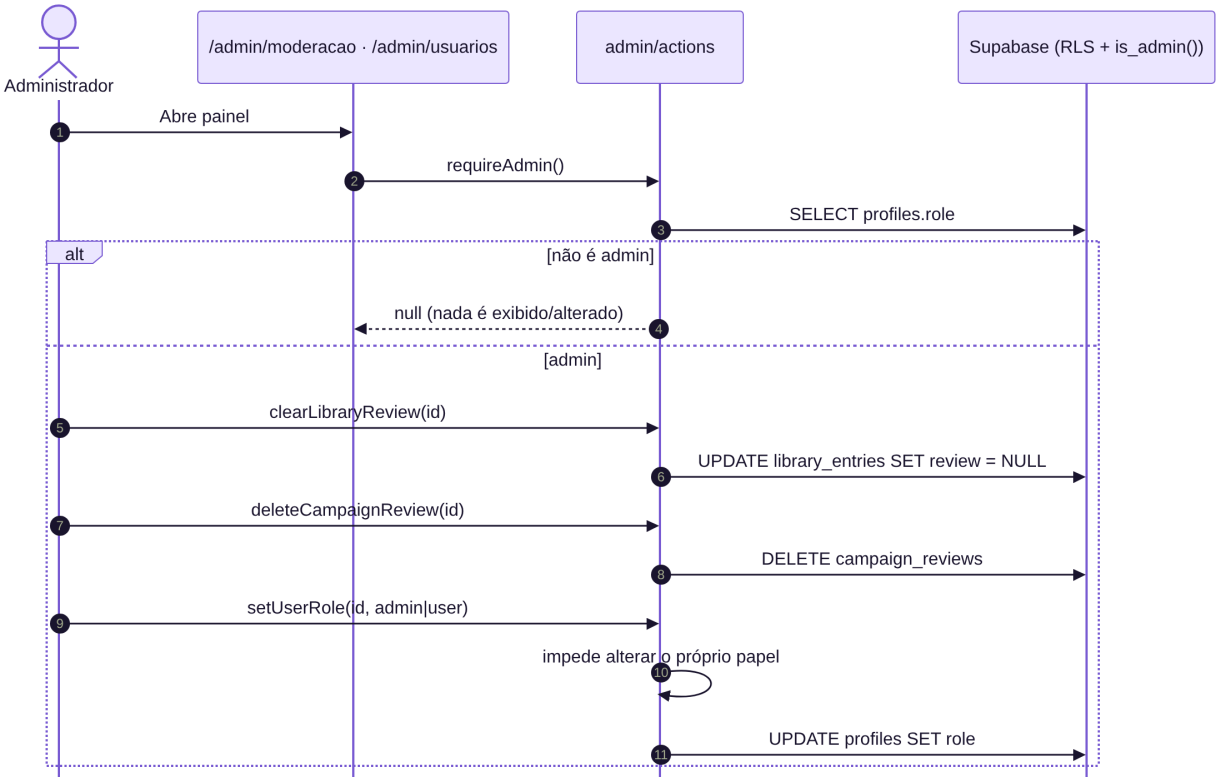
Sequência — campanha

8.5 Importar obra de API externa (UC14)



Sequência — importar obra

8.6 Moderação e papéis (UC15, UC16)



Sequência — moderação

9. Server Actions e Integrações

9.1 Server Actions

Módulo	Action	Entrada	Autorização	Efeito
`(auth)/actions.ts`	`signUp`	username, email, password	—	Cria usuário e sessão.
	`signIn`	email, password, next	—	Cria sessão; redireciona para `next` interno.
	`signOut`	—	sessão	Encerra sessão.
`obras/actions.ts`	`saveEntry(mediaId)`	status?, progress?, rating? (1–10), review?, spoilers?	sessão + RLS	Upsert em `library_entries`.
`listas/actions.ts`	`createList`	title, description, is_public	sessão	Insert em `lists`.
	`deleteList`	id	dono	Delete em `lists`.
	`removeListItem`	list_id, media_id	dono (RLS)	Delete em `list_items`.
	`updateProfile`	bio, location, avatar_url, banner_url	próprio	Update em `profiles`.
`lib/actions/lists.ts`	`getMyListsForMedia`	mediaId (UUID)	sessão	Listas + flag `hasMedia`.
	`setMediaInList`	listId, mediaId, add	dono	Insert/Delete em `list_items`.
	`createListWithMedia`	title, isPublic, mediaId	sessão	Cria lista e adiciona obra.
`campanhas/actions.ts`	`createCampaign`	campos da campanha	sessão	Insert em `campaigns`.
	`submitCampaignReview(id)`	rating 0–5, comment	sessão	Upsert em `campaign_reviews`.
	`updateCampaign` / `deleteCampaign`	id + campos	autor ou admin	Update/Delete em `campaigns`.
`admin/actions.ts`	`createMedia` / `updateMedia` / `deleteMedia`	campos da obra	admin	CRUD em `media`.
	`importExternalMedia`	type, source, external_id	admin	Busca na API e upsert em `media`.

Módulo	Action	Entrada	Autorização	Efeito
	`clearLibraryReview`	id	admin	`review = null`.
	`deleteCampaignReview`	id	admin	Delete em `campaign_reviews`.
	`setUserRole`	id, role	admin (não a si mesmo)	Update em `profiles.role`.

Todas as actions que alteram dados chamam `revalidatePath` para atualizar as páginas afetadas.

9.2 Respostas tipadas (listas)

```
type MutationResult =
  | { status: "ok" }
  | { status: "login" }
  | { status: "error"; message: string };
```

9.3 APIs externas

API	Tipos	Endpoint	Autenticação	Cache
AniList	anime, mangá	`POST https://graphql.anilist.co`	Nenhuma	1 h
Jikan v4	anime, mangá (*fallback*)	`GET https://api.jikan.moe/v4/{anime|manga}`	Nenhuma (3 req/s, 60 req/min)	1 h / 24 h detalhe
TMDB v3	filme, série	`https://api.themoviedb.org/3`	`TMDB_API_READ_TOKEN` (Bearer) ou `TMDB_API_KEY`	1 h

Normalização: todas as respostas viram `ExternalMedia` (título, subtítulo — estúdio/autor/emissora —, sinopse sem HTML, capa, ano, unidades e gêneros traduzidos para pt-BR, sem duplicatas).

Resiliência (`fetchJson`) - Timeout de 15 s via `AbortSignal.timeout`. - Uma nova tentativa após HTTP 429 (respeitando `Retry-After`, até 5 s) ou falha de rede (espera de 1,1 s). - Erros viram `ExternalApiError`, que aciona o *fallback* AniList → Jikan. - `searchAllExternal` usa `Promise.allSettled`: um tipo com falha é reportado em `failed` sem derrubar os demais.

9.4 Variáveis de ambiente

Variável	Onde	Obrigatória	Descrição
`NEXT_PUBLIC_SUPABASE_URL`	cliente + servidor	Sim (para dados reais)	URL do projeto Supabase.
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	cliente + servidor	Sim (para dados reais)	Chave pública *anon*.
`TMDB_API_READ_TOKEN`	servidor	Não	Token v4 do TMDB (prioritário).

Variável	Onde	Obrigatória	Descrição
`TMDB_API_KEY`	servidor	Não	Chave v3 do TMDB (alternativa).

10. DevOps: CI/CD, Docker e Deploy

10.1 Diagrama de implantação

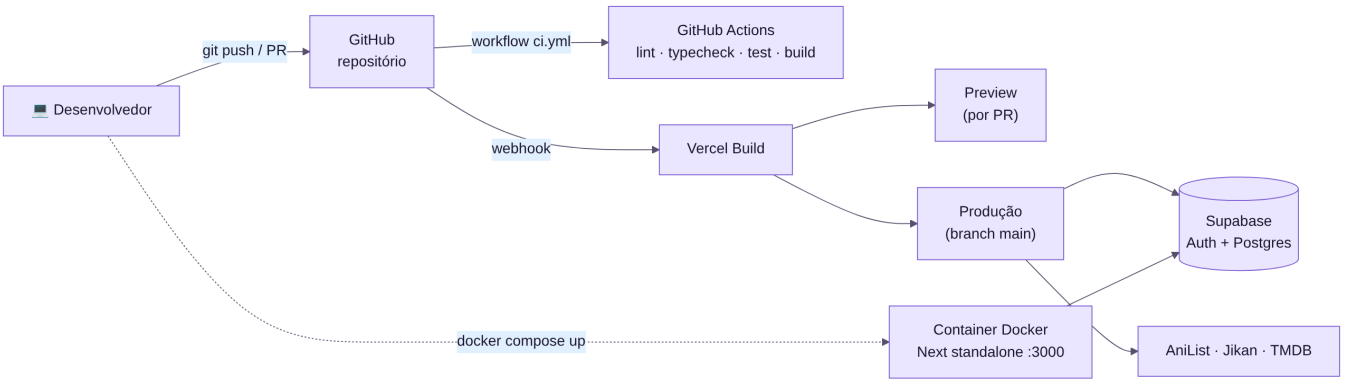


Diagrama de implantação

10.2 Integração contínua (GitHub Actions)

Arquivo: `.github/workflows/ci.yml` — executa em *push* e *pull request* para `main`, com `concurrency` que cancela execuções antigas da mesma branch.

Job	Depende de	Etapas
build-and-test	—	`npm ci` → lint → typecheck → testes unitários → testes de integração → build → `npm audit` (não bloqueante)
e2e	build-and-test	Instala o Chromium → build em modo demonstração → Playwright → publica `playwright-report`
performance	build-and-test	Build → sobe o servidor → perfis *smoke* e *load* → publica resultados

Os jobs **e2e** e **performance** rodam em paralelo. Sem as variáveis do Supabase, o build usa o catálogo de demonstração, garantindo dados previsíveis e nenhum acesso ao banco real.

10.3 Entrega contínua (Vercel)

- O repositório é importado na Vercel (framework **Next.js**, comando `npm run build`).
- Cada *pull request* gera um **deploy de preview** com URL própria.
- O *merge* na `main` publica em **produção**.
- Variáveis de ambiente são configuradas no painel da Vercel (*Settings* → *Environment Variables*).
- A central de documentação fica disponível em `/docs` no mesmo deploy.

10.4 Docker

- `Dockerfile` *multi-stage*: `deps` (npm ci) → `builder` (`next build` com `output: "standalone"`) → `runner` (`node:22-alpine`, usuário `nextjs` sem privilégios, porta 3000).
- `docker-compose.yml`: serviço `app`, variáveis do `.env.local`, `restart: unless-stopped` e *healthcheck* HTTP.
- `.dockerignore` exclui dependências, artefatos de build e arquivos de ambiente do contexto.

```
docker compose --env-file .env.local up --build
```

10.5 Execução local

```
npm install
cp .env.example .env.local   # preencher Supabase (e TMDB opcional)
npm run dev                  # http://localhost:3000
```

Banco: executar `supabase/schema.sql` e depois as migrações `002` e `003` no SQL Editor do Supabase. Para tornar um usuário admin:

```
update profiles set role = 'admin' where username = 'seu_usuario';
```

10.6 Fluxo de trabalho Git

1. Criar *branch* a partir da `main` (`feat/…`, `fix/…`, `docs/…`).
2. Commits pequenos e descritivos (Conventional Commits: `feat:`, `fix:`, `docs:`, `build:`).
3. Abrir *pull request* → CI + preview na Vercel.
4. Revisão e *merge* → produção.

10.7 Segurança operacional

- Segredos somente em variáveis de ambiente (Vercel, GitHub, `.env.local`).
- Nenhuma chave `service_role` no projeto: todo acesso usa a chave *anon* + RLS.
- Contêiner sem *root*; permissões mínimas no workflow (`contents: read`).

11. Estratégia de Testes

11.1 Pirâmide

Nível	Ferramenta	Local	Situação
Unitário	Vitest (projeto `unit`)	`src/**/*.test.ts`, `tests/unit/`	Implementado
Integração	Vitest (projeto `integration`)	`tests/integration/`	Implementado
End-to-end	Playwright	`tests/e2e/`	Implementado
Performance	Script Node	`tests/performance/load-test.mjs`	Implementado
Banco/RLS	SQL no Supabase	—	Manual (roteiro abaixo)

11.2 Suítes

Suíte	O que verifica
Unitários	Filtros, ordenação e paginação da busca; `timeAgo`; rótulos por tipo; médias; `canManageCampaign`; cliente HTTP (timeout, retry, 429); normalização AniList/Jikan/TMDB e *fallback*; links e arquivos da documentação.
Integração	Server Actions de autenticação, biblioteca, listas, campanhas e admin com um dublê do Supabase (`tests/helpers/supabase.ts`); middleware de sessão; `data.ts`; pipeline catálogo → busca.
E2E	Home, busca pela barra do topo, filtros, ordenação, paginação, login/cadastro, campanhas e página 404 no navegador.
Performance	Latência p50/p95/p99, vazão e taxa de erros nos perfis *smoke* (1 conexão), *load* (10) e *stress* (50), com limites que reprovam a CI.

Comandos: `npm test` (unitários + integração), `npm run test:unit`, `npm run test:integration`, `npm run test:e2e`, `npm run test:perf`.

11.3 Casos de teste manuais (aceitação)

ID	Cenário	Passos	Resultado esperado
CT01	Cadastro com senha curta	Cadastrar com senha “123”	Mensagem de erro; nenhuma conta criada.
CT02	Rota protegida	Acessar `/minha-biblioteca` deslogado	Redireciona para `/login?next=/minha-biblioteca`.

ID	Cenário	Passos	Resultado esperado
CT03	Login com retorno	Fazer login a partir do CT02	Volta para `/minha-biblioteca`.
CT04	Avaliar obra	Dar nota 8 em uma obra	`rating = 4.0`; média atualizada.
CT05	Spoiler	Resenha com “contém spoilers”	Texto oculto para outros usuários até clicar.
CT06	Lista privada	Criar lista privada e acessar com outra conta	Lista não visível.
CT07	Obra duplicada na lista	Adicionar a mesma obra duas vezes	Sem erro, sem duplicata.
CT08	Campanha de terceiro	Tentar editar campanha de outro usuário	Operação ignorada/bloqueada.
CT09	Avaliar campanha 2×	Avaliar, depois avaliar de novo	Mantém só a última avaliação.
CT10	Painel admin sem permissão	Usuário comum abre `/admin`	Acesso negado.
CT11	Importação repetida	Importar a mesma obra duas vezes	Uma única obra; edições preservadas.
CT12	AniList fora do ar	Bloquear `graphql.anilist.co` e buscar anime	Resultados vindos do Jikan.
CT13	Moderação	Limpar resenha	Texto removido, nota mantida.
CT14	Auto-rebaixamento	Admin tenta tirar o próprio papel	Nada muda.
CT15	Documentação	Acessar `/docs` e baixar Word/PDF	Arquivos abrem corretamente.

11.4 Critérios de qualidade

- CI verde (lint, typecheck, testes e build) obrigatório para *merge*.
- Toda regra de negócio nova em `lib/` deve vir acompanhada de teste unitário.